

# Proximal Policy Optimization (PPO)

CS 5624: Natural Language Processing

*Fall 2026*

<https://tuvllms.github.io/nlp-fall-2026/>

Tu Vu



# Logistics

- Final projects
  - Project proposal
    - Template available on Piazza @23, due on 10/12 (no late days)
    - Potential topics shared on Piazza @24
  - Sample project available on Piazza @21
- HW1 due tomorrow Friday 10/9

# AI news

# Decision models (TypeSafe's Jev-style)



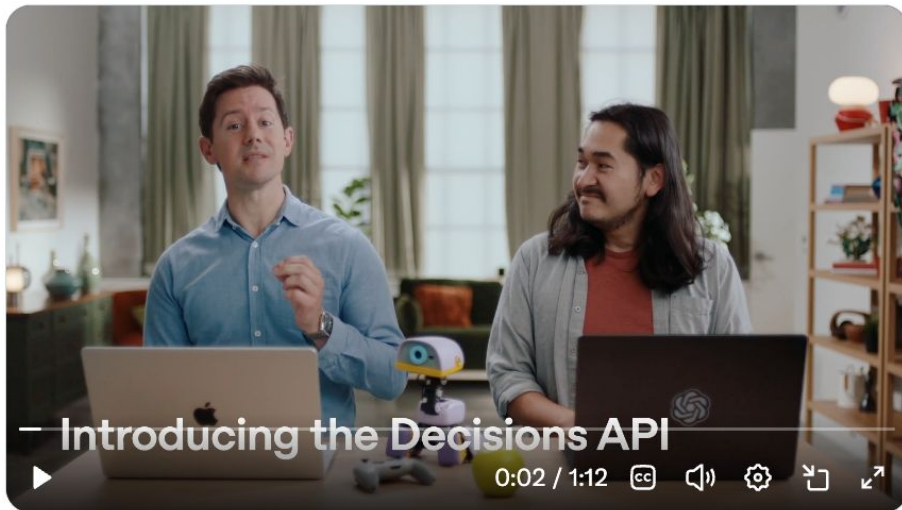
OpenAI Developers

@OpenAIDevs



Let your app choose the right model, tool, or action in near real-time with Decisions API, now available to all developers in public beta.

The Decisions API makes decisions up to 10x faster than GPT-6 Luna through the Responses API.



4:47 PM · Oct 6, 2026 · 1.7M Views



Unslot AI 🟡 @UnslotAI · 22h



You can now train your own Decision model like Jev locally!

We increased Qwen3.5 0.8B's aggregate accuracy from 20.7% to 74.3% across 3 decision benchmarks - on just 4GB VRAM.

Turn any LLM like Qwen3.8, Gemma 4 into decision models with our open-source Unslot repo.

We

[Show more](#)

## Train your own Decision Model

You can now train your own decision model like **Jev**, **Laya** and **Clef** with Unslot.

Fine-tune [Qwen3.8](#) and [Gemma 4](#) LLMs to convert them into decision models so they **make decisions** instead of generating text. The fine-tuned model reads your input, scores the options you provide and returns a choice with a probability. Training uses the same dataset format as Laya and Clef.

| Model        | type-decision    | BANKING77       | CLINC150         | Holdout acc |
|--------------|------------------|-----------------|------------------|-------------|
| Qwen3.5-0.8B | 36% → <b>73%</b> | 7% → <b>74%</b> | 19% → <b>76%</b> | 78%         |
| Qwen3.5-2B   | 33% → <b>78%</b> | 1% → 58%        | 1% → 62%         | 81%         |

We fine-tuned LLMs with a Clef head using LoRA ( $r=64$ ) for just one epoch, boosting downstream accuracy from 30-37% (which is roughly chance level) to **high as 78%**. Test sets were **decontaminated** against the training data.

Prompt Qwen3.5-0.8B



"Hi, I was charged twice for invoice #4411. Please refund the duplicate today."

Before

This looks like a billing issue since the customer was charged twice. They are asking for a refund of the duplicate charge. It seems fairly urgent because they want it resolved today, so I'd suggest routing it to the billing team with high priority.

After fine-tuning

| Team           | Probability                        |
|----------------|------------------------------------|
| <b>Billing</b> | Technical 5% - Sales 2% <b>93%</b> |
| Refund         |                                    |
| <b>Yes</b>     | No 25% <b>75%</b>                  |
| Urgency        |                                    |
| <b>Today</b>   | Soon 7% - Not urgent 1% <b>92%</b> |

💬 121

↻ 487

❤ 3.5K

📊 176K



 IdeaLens

## Is this text human or AI?

We check its  ideas and  prose separately.

 **Important:** This is a research demo, and we save your text and prediction for analysis. Errors are rare but possible. 

 Human  Human

 Human  AI

 AI  Human

 AI  AI

Paste your text

0 / 12,000 words

 **Before you check: what's your prediction?** Optional

 Ideas

Human

AI

Mixed

 Prose

Human

AI

Mixed

 about 15–60 seconds

Check this text

## Ideas from a person

Prose by a person

 Q1 Human ideas, human prose **shared provenance**

human-written documents, 19 evaluations



## Ideas from an AI

 Q2 AI ideas, human prose **mixed provenance**

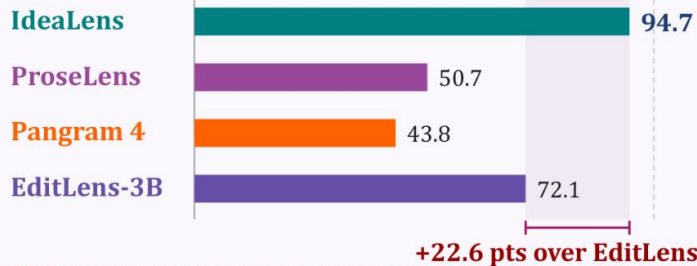
people rewrite 50 AI stories from their outlines



Prose by an AI

 Q4 Human ideas, AI prose **mixed provenance**

AI polishes, rewrites or writes up a person's ideas, 12 evaluations



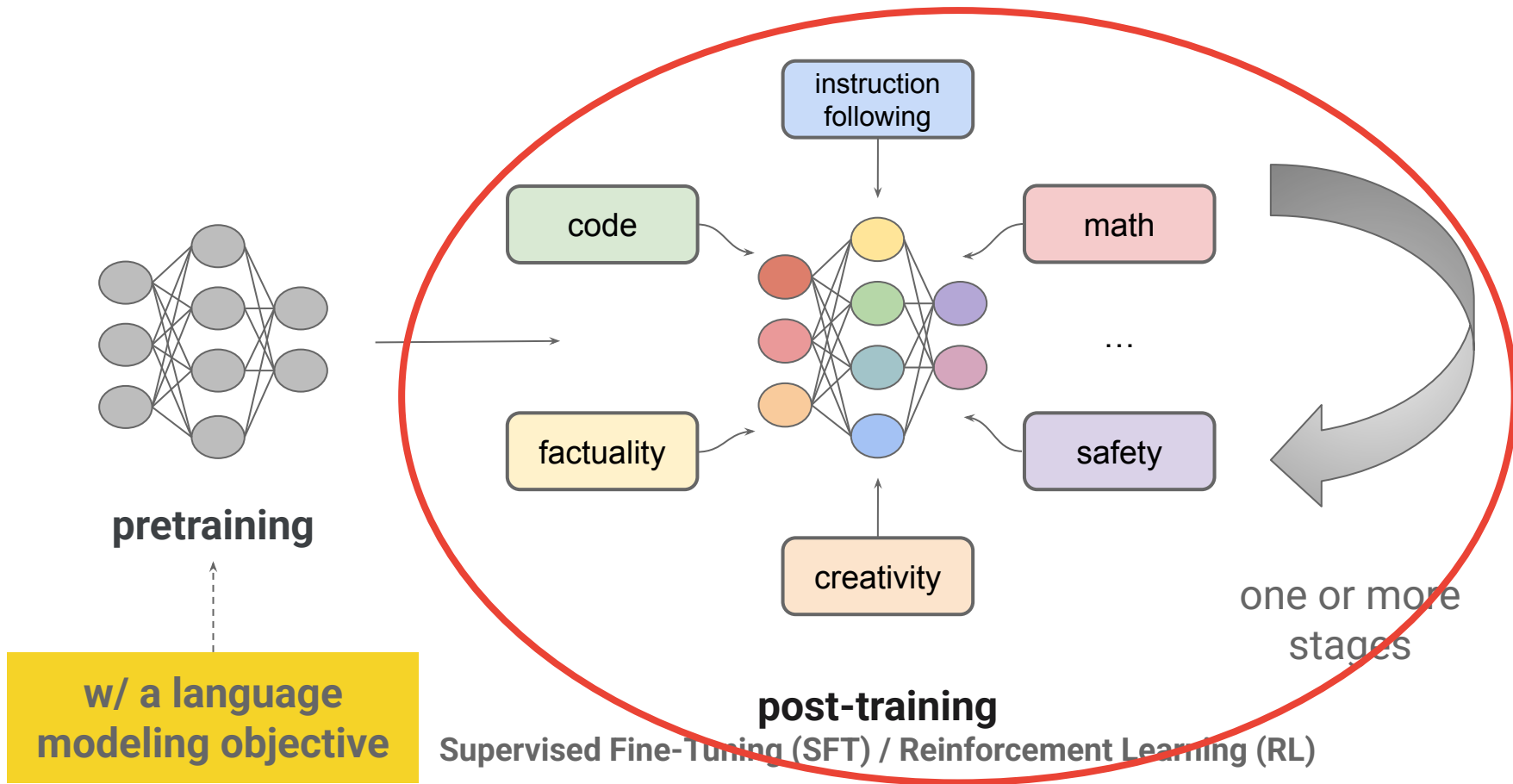
 Q3 AI ideas, AI prose **shared provenance**

AI-written documents, 19 evaluations



Accuracy of detection based on the source of ideas

# Where are we?



# Reinforcement learning from human feedback (RLHF)

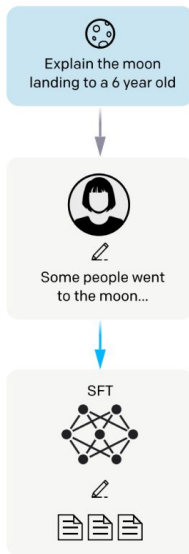
## Step 1

**Collect demonstration data, and train a supervised policy.**

A prompt is sampled from our prompt dataset.

A labeler demonstrates the desired output behavior.

This data is used to fine-tune GPT-3 with supervised learning.



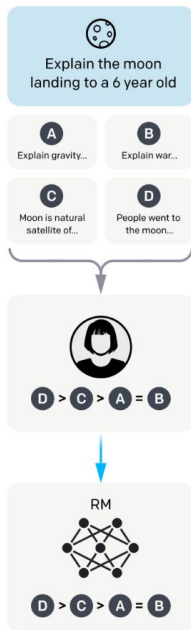
## Step 2

**Collect comparison data, and train a reward model.**

A prompt and several model outputs are sampled.

A labeler ranks the outputs from best to worst.

This data is used to train our reward model.



## Step 3

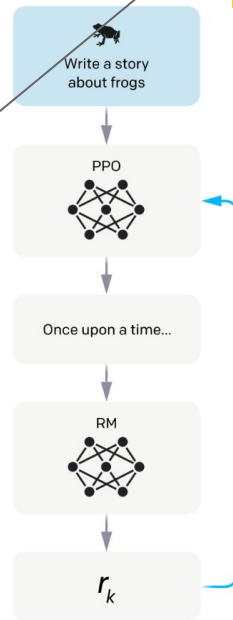
**Optimize a policy against the reward model using reinforcement learning.**

A new prompt is sampled from the dataset.

The policy generates an output.

The reward model calculates a reward for the output.

The reward is used to update the policy using PPO.



**Learn from data generated by the current policy**

## Step 3: RL training

The second term prevents the model from deviating too far from the distribution on which the reward model is accurate.

$y \sim \pi_\theta(y|x)$   
rollout

$$\max_{\pi_\theta} \mathbb{E}_{x \sim D, y \sim \pi_\theta(y|x)} [r_\phi(x, y)] - \beta D_{KL} [\pi_\theta(y|x) || \pi_{\text{ref}}(y|x)]$$

where  $\beta$  is a parameter controlling the deviation from the base reference policy  $\pi_{\text{ref}}$ , namely the initial SFT model  $\pi^{SFT}$ . In practice, the language model policy  $\pi_\theta$  is also initialized to  $\pi^{SFT}$ .

# Expectation

The **expectation** (or expected value) of a random variable  $X$  is

$$\mathbb{E}[X] = \sum_x x P(X = x)$$

where:

- $x$  is a possible value of  $X$
- $P(X = x)$  is the probability of that value
- $\sum_x$  means "sum over all possible values of  $X$ "

So expectation is simply a **probability-weighted average**.

**Example: fair six-sided die**

Let  $X$  be the number rolled. Each outcome has probability  $1/6$ :

$$\begin{aligned}\mathbb{E}[X] &= 1 \left(\frac{1}{6}\right) + 2 \left(\frac{1}{6}\right) + \cdots + 6 \left(\frac{1}{6}\right) \\ &= \frac{1 + 2 + 3 + 4 + 5 + 6}{6} \\ &= \boxed{3.5}.\end{aligned}$$

This does **not** mean you can roll a 3.5. It means that the average roll approaches 3.5 over many rolls.

**An expectation is a weighted average over all possible outcomes, where each outcome is weighted by how likely it is.**

Assume two different distributions for predicting the next word:

- $P$  (from Model 1):
  - $mat \rightarrow 0.7$
  - $floor \rightarrow 0.2$
  - $chair \rightarrow 0.1$
- $Q$  (from Model 2):
  - $mat \rightarrow 0.5$
  - $floor \rightarrow 0.3$
  - $chair \rightarrow 0.2$

## Kullback–Leibler (KL) Divergence Calculation

KL divergence measures how much  $P$  diverges from  $Q$ :

$$D_{KL}(P||Q) = \sum_i P(i) \log \frac{P(i)}{Q(i)}$$

Substituting the values:

$$D_{KL}(P||Q) = 0.7 \log \frac{0.7}{0.5} + 0.2 \log \frac{0.2}{0.3} + 0.1 \log \frac{0.1}{0.2}$$

# Policy Gradient Algorithms

- We optimize expected reward:

$$J(\theta) = \mathbb{E}_{y \sim \pi_{\theta}(\cdot|x)}[R(x, y)].$$

$$J(\theta) = \sum_y \pi_{\theta}(y|x) R(x, y).$$

**Consider every possible completion  $y$ , multiply its reward by the probability that the model generates it, and then add everything together.**

$$\pi_{\theta}(y \mid x) = \prod_{t=1}^T \pi_{\theta}(y_t \mid x, y_{<t})$$

Diagram illustrating the relationship between action<sub>t</sub> and state<sub>t</sub> in the policy function. The action<sub>t</sub> (represented by a yellow box) points to  $y_t$ , and the state<sub>t</sub> (represented by a yellow box) points to the circled term  $x, y_{<t}$ .

$$\log \pi_{\theta}(y \mid x) = \log \prod_{t=1}^T \pi_{\theta}(y_t \mid x, y_{<t}) = \sum_{t=1}^T \log \pi_{\theta}(y_t \mid x, y_{<t}).$$

# Policy Gradient: advantage form

In modern algorithms:

$$\nabla_{\theta} J(\theta) = \mathbb{E} \left[ \sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) A_t \right]$$

$A_t$  (or  $\Psi_t$ )

Learning signal.

- Positive  $\rightarrow$  action better than expected  $\rightarrow$  reinforce.
- Negative  $\rightarrow$  worse  $\rightarrow$  suppress.
- Zero  $\rightarrow$  no update.

**Policy gradient updates the model by increasing the log-probability of actions that produced better-than-expected outcomes and decreasing those that produced worse outcomes.**

In SFT, the token-level loss is

$$\mathcal{L}_{\text{SFT}} = - \sum_t \log \pi_{\theta}(y_t \mid y_{<t}, x).$$

You can think of every demonstrated token as having weight 1.

In vanilla policy-gradient RL, the corresponding objective is approximately

$$\mathcal{L}_{\text{RL}} = - \sum_t A_t \log \pi_{\theta}(a_t \mid a_{<t}, x),$$

where  $a_t$  is the sampled token and  $A_t$  is its advantage. So the gradient looks like a **weighted NLL gradient**:

$$\nabla_{\theta} \mathcal{L}_{\text{RL}} = - \sum_t A_t \nabla_{\theta} \log \pi_{\theta}(a_t \mid a_{<t}, x).$$

There is one important distinction from SFT. **SFT trains on target tokens supplied by the dataset, whereas RL usually trains on tokens sampled from the policy itself.**

## SFT

The 5 tokens come from a **human/reference response**:

$$o = (o_1, \dots, o_5) \sim \text{human data.}$$

We maximize

$$J_{\text{SFT}}(\theta) = \frac{1}{5} \sum_{t=1}^5 \log \pi_{\theta}(o_t \mid o_{<t})$$

So the model is told:

“Make the human’s tokens more likely.”

## RL

The 5 tokens are **sampled from the model**:

$$o = (o_1, \dots, o_5) \sim \pi_{\theta_{\text{old}}}(\cdot \mid x)$$

Then we evaluate the generated response, get reward  $r$ , and derive advantage  $A$ .

Conceptually, the objective is

$$J_{\text{RL}}(\theta) = \frac{1}{5} \sum_{t=1}^5 A \log \pi_{\theta}(o_t \mid x, o_{<t})$$

Thus:

$A > 0 \Rightarrow$  make the model’s sampled tokens more likely

and

$A < 0 \Rightarrow$  make the model’s sampled tokens less likely.

# High-level framework (cont'd)

- **Return**  $G_t$ : "how good the outcome was from time  $t$  onward."
  - In single-final-reward RLHF,  $G_t = R$  for all tokens.
- **Baseline**  $b_t$ : something that you subtract to reduce variance.
  - Baseline does not change the expected gradient, but it reduces noise.
- **Advantage**  $A_t$ : "better-than-expected"

$$A_t = G_t - b_t.$$

**Key takeaway:** in single-final-reward RLHF, most differences come from *how you choose  $b_t$  and how you constrain updates.*

Think of  $G_t$  as the score from your teacher  
for each essay you write

Think of  $b_t$  as your own internal expectation  
("I usually get an 80 on essays like this")

# Return

## General discounted return

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}.$$

If the episode terminates at time  $T$ , this becomes

$$G_t = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{T-t-1} R_T.$$

When  $\gamma = 1$

$$G_t = R_{t+1} + R_{t+2} + \dots + R_T.$$

In RLHF with a single terminal reward

$$R_1 = 0, \dots, R_{T-1} = 0, \quad R_T = R.$$

So for every timestep  $t$ ,

$$G_t = R.$$

The return is the total future reward associated with a token/action.

$R_{t+1}$  : the reward received after taking the action at time  $t$

# KL penalty per token (reward shaping)

RLHF usually adds a per-token KL penalty to keep the policy close to a reference model.

So per-token reward becomes:

$$r_t = -\beta \text{KL}_t, \quad r_T = R(x, y) - \sum_t \beta \text{KL}_t.$$

## KL penalty per token (reward shaping)

$$R_{\text{shaped}} = R_{\text{model}} - \beta \sum_t \text{KL}_t$$

So the KL is computed **per token**, then summed over the sequence.

$$D_{\text{KL}}(P_t \| Q_t) = \sum_{a \in \mathcal{V}} P_t(a) \log \frac{P_t(a)}{Q_t(a)}$$

**KL divergence:** measures the difference between the policy and reference distributions.

$$R_{\text{shaped}} = R_{\text{model}} - \beta \sum_t \log \frac{P_t(a_t)}{Q_t(a_t)}, \quad a_t \sim P_t$$

**Actual sampled reward:** penalizes the log-probability difference for each action  $a_t$  that the policy takes.

$$A_t = G_t - b_t$$

$$= R_{\text{shaped}} - b_t$$

$$= R_{\text{model}} - \beta \sum_t \log \frac{P_t(a_t)}{Q_t(a_t)} - b_t$$

## Policy Gradient Algorithms (cont'd)

- REINFORCE
- RLOO (REINFORCE Leave One Out)
- PPO (Proximal Policy Optimization)

| Method    | $G_t$ | $b_t$      | Advantage shape     |
|-----------|-------|------------|---------------------|
| REINFORCE | $R$   | batch mean | same for all tokens |
| RLOO      | $R$   | group mean | same for all tokens |
| PPO       | $R$   | $V(s_t)$   | different per token |

- **REINFORCE**  $\rightarrow A_t = G_t - b$
- **RLOO**  $\rightarrow A_t = R - \text{leave-one-out baseline}$
- **PPO**  $\rightarrow A_t$  from value function / GAE + clipped ratio

# REINFORCE

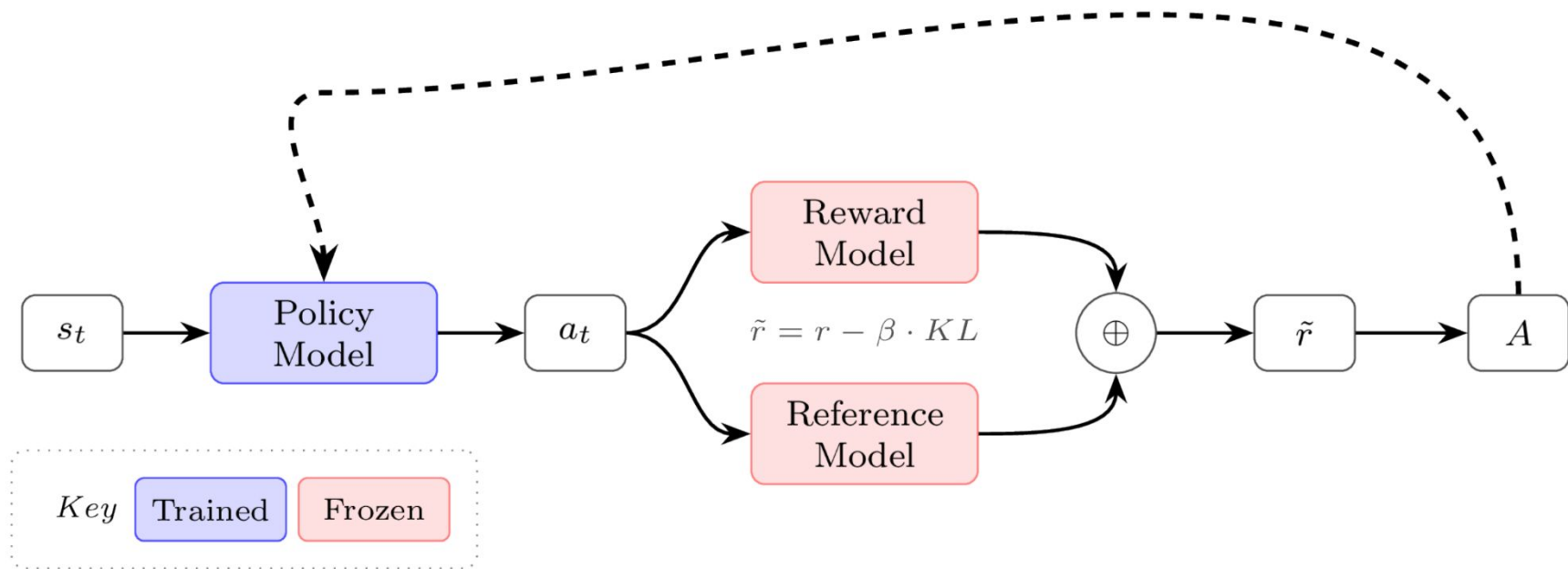


Figure 17: Basic REINFORCE architecture for language models. The shaped reward combines the reward model score with a KL penalty from the reference model. We build on this structure throughout the chapter.

## REINFORCE (cont'd)

After computing the shaped reward (a single scalar), REINFORCE uses:

$$A_t = R_{\text{shaped}} - b.$$

So:

- Even though KL was per-token,
- It is **aggregated into one scalar reward**,
- Then that scalar is broadcast to all tokens.

# REINFORCE (cont'd)

$b_t = \text{mean reward over batch.}$

So if your batch has rewards  $[R_1, R_2, R_3]$ ,

$$b_t = \frac{R_1 + R_2 + R_3}{3}.$$

This baseline is the same for all tokens in that sequence.

Then:

$$G_t - b_t = R - \text{batch mean.}$$

So every token in that completion gets the same advantage.

# REINFORCE (cont'd)

## Intuition

- Sample a completion.
- If reward is high, reinforce every token in that completion.
- If reward is low, suppress every token in that completion.

## Update shape (minimal form)

$$\sum_t \nabla_{\theta} \log \pi_{\theta}(y_t | x, y_{<t}) (R - b)$$

## REINFORCE (cont'd)

It is a **Monte Carlo** gradient estimator: it uses sampled outcomes.

With a single final reward, credit is **broadcast** to all tokens.

Main weakness: **high variance**, because one scalar reward must explain many tokens.

# RLOO

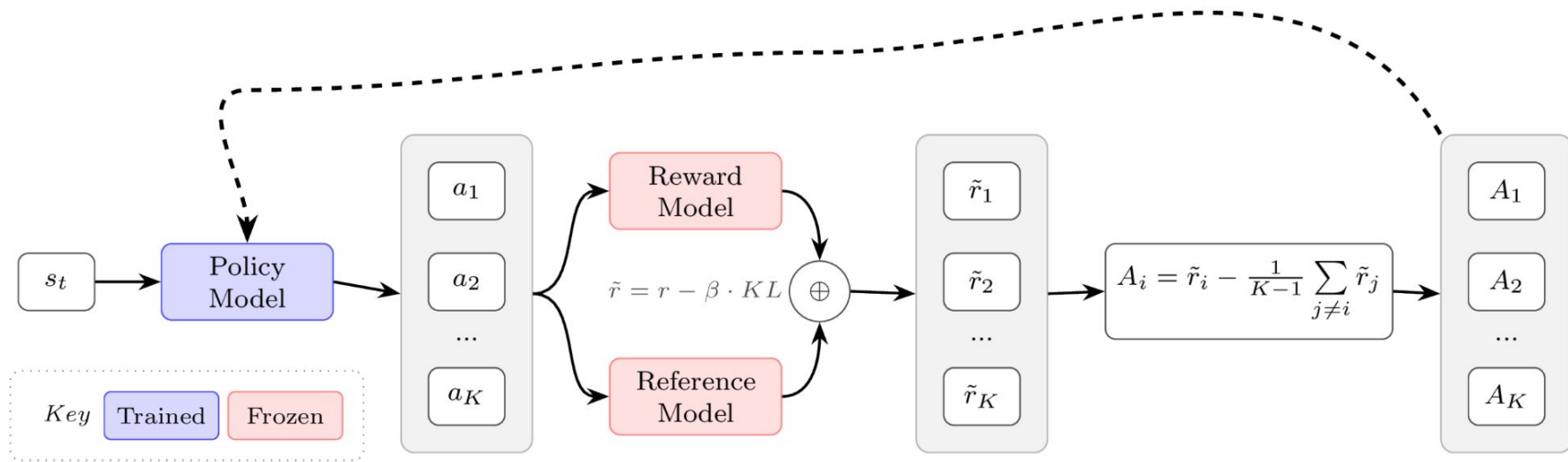


Figure 18: REINFORCE Leave-One-Out (RLOO) architecture. Multiple completions per prompt provide a leave-one-out baseline for advantage estimation without learning a value function.

## RLOO (cont'd)

If you generate multiple completions per prompt:

$b$  = mean reward of other completions.

Still one scalar per completion.

Still identical across tokens.

# RLOO (cont'd)

## Motivation

REINFORCE is noisy because the baseline is crude (often a batch average). RLOO improves the baseline by making it **prompt-conditional**.

## Setup

- For one prompt  $x$ , sample  $K$  completions  $y^{(1)}, \dots, y^{(K)}$ .
- Score each completion:  $R^{(k)} = R(x, y^{(k)})$ .

## Baseline

For completion  $k$ , the baseline is the mean reward of the *other* completions:

$$b^{(k)} = \frac{1}{K-1} \sum_{i \neq k} R^{(i)}.$$

## Advantage

$$A^{(k)} = R^{(k)} - b^{(k)}.$$

## RLOO (cont'd)

RLOO computes:

$$A^{(k)} = R_{\text{shaped}}^{(k)} - \text{mean of other rewards.}$$

Again:

- Shaped reward includes summed KL.
- But the advantage is one scalar per completion.
- That scalar is broadcast to all tokens.

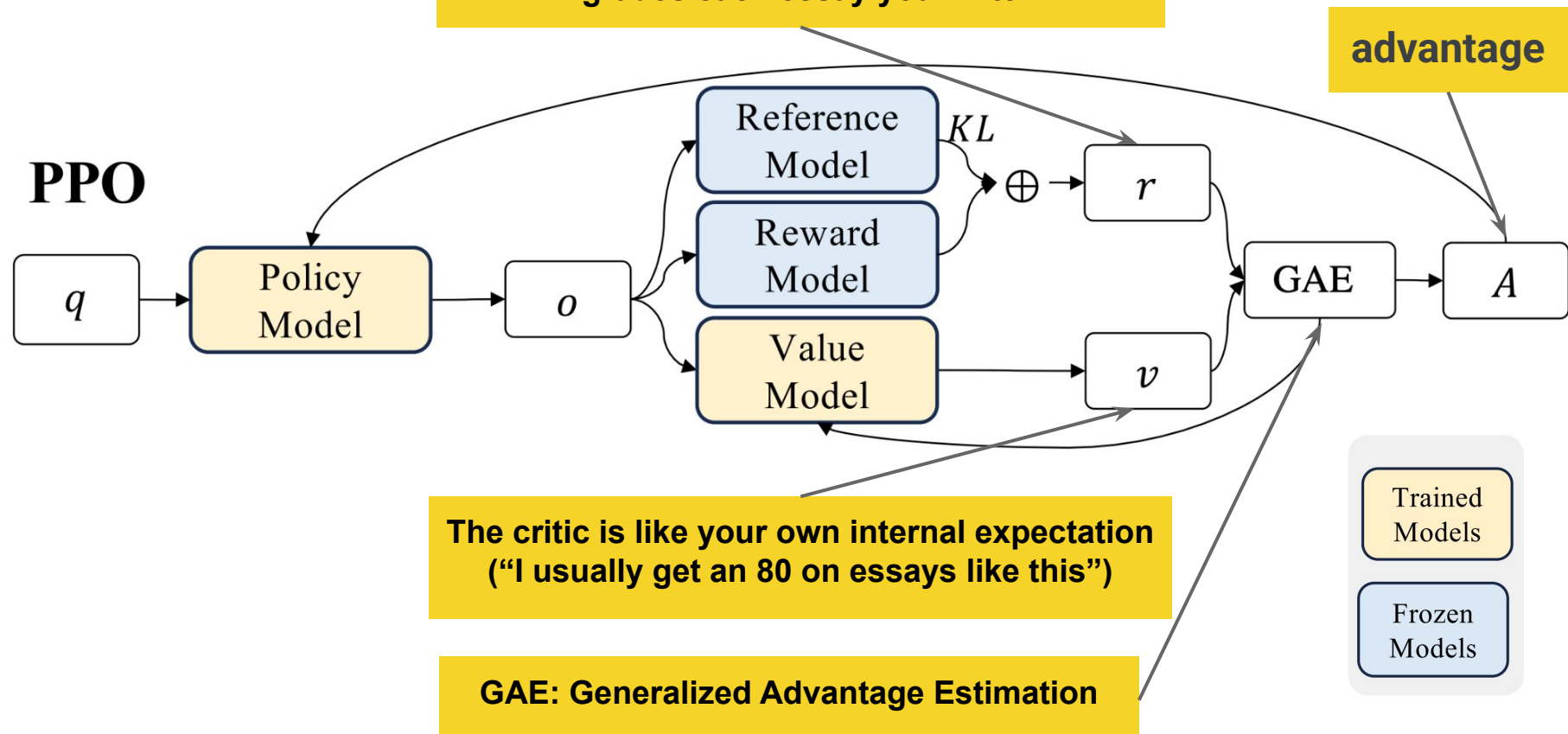
So still no token-level differentiation.

- RLOO keeps the simplicity of REINFORCE.
- It reduces variance because it compares completions **within the same prompt**, which removes prompt difficulty effects.
- It still assigns a **sequence-level** advantage that is broadcast to tokens.

## Quick intuition line

RLOO turns "is this good?" into "is this better than the other tries for the same question?"

# PPO



## Advantage

$$A_t = G_t - V(s_t)$$

$$V_{\phi}(s_t) \approx \mathbb{E}_{\pi}[G_t \mid s_t]$$

$$A_t = \underbrace{G_t}_{\text{what actually happened}} - \underbrace{\mathbb{E}[G_t \mid s_t]}_{\text{what we expected}}$$

## Step 1: What is $G_t$ ?

The return is all future rewards:

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \gamma^3 r_{t+4} + \dots$$

Pull out the first reward:

$$G_t = r_{t+1} + \gamma (r_{t+2} + \gamma r_{t+3} + \gamma^2 r_{t+4} + \dots)$$

Look at the part inside the parentheses:

$$\underbrace{r_{t+2} + \gamma r_{t+3} + \gamma^2 r_{t+4} + \dots}_{G_{t+1}}$$

So exactly:

$$G_t = r_{t+1} + \gamma G_{t+1}$$

## Step 2: But we don't know $G_{t+1}$ yet

At time  $t$ , the future hasn't happened yet.

That's where the value function comes in. Remember:

$$V(s_{t+1}) \approx \mathbb{E}[G_{t+1} \mid s_{t+1}]$$

In words:

$V(s_{t+1})$  predicts the return that we'll get starting from the next state.

Therefore, we approximate

$$G_{t+1} \approx V(s_{t+1})$$

and consequently:

$$G_t \approx r_{t+1} + \gamma V(s_{t+1})$$

Now plug this into our advantage equation:

$$A_t = G_t - V(s_t)$$

giving

$$A_t \approx r_{t+1} + \gamma V(s_{t+1}) - V(s_t)$$

which is the **TD error**.

### **In our LLM RL setting**

We use

$$r_{t+1} = 0, \quad \gamma = 1$$

so the TD error simplifies to

$$A_t \approx V(s_{t+1}) - V(s_t)$$

In words, the advantage measures **how much the value increases from the current state to the next state**.

# 1) What the value model learns

Value model predicts a scalar

$$V(s_t) \approx \mathbb{E}[G_t \mid s_t]$$

where the "state" is the prefix:

$$s_t = (x, a_1, \dots, a_{t-1})$$

So input is **prompt + previously generated tokens**.

Output is **one number per token position**.

## 2) Collect training data (rollouts)

Pick a prompt:

x: Translate to French: "Hello world"

Sample a completion with the current policy:

y: "Bonjour le monde"

Tokens:  $a_1 = \text{Bonjour}$ ,  $a_2 = \text{le}$ ,  $a_3 = \text{monde}$ .

Now you have prefix states:

- $s_1 = \text{prompt}$
- $s_2 = \text{prompt} + \text{"Bonjour"}$
- $s_3 = \text{prompt} + \text{"Bonjour le"}$

These are your **training inputs** to the value model.

$$R = 0.9$$

Define rewards per step (simplest RLHF case, no KL):

- $r_1 = 0$
- $r_2 = 0$
- $r_3 = 0.9$  (terminal)

(If you include KL, then  $r_t$  also has  $-\beta \text{KL}_t$  terms.)

So the supervised dataset looks like:

| input prefix $s_t$  | label $G_t$ |
|---------------------|-------------|
| prompt              | 0.9         |
| prompt + Bonjour    | 0.9         |
| prompt + Bonjour le | 0.9         |

$$R = 0.9$$

Define rewards per step (simplest RLHF case, no KL):

- $r_1 = 0$
- $r_2 = 0$
- $r_3 = 0.9$  (terminal)

(If you include KL, then  $r_t$  also has  $-\beta \text{KL}_t$  terms.)

So the supervised dataset looks like:

| input prefix $s_t$  | label $G_t$ |
|---------------------|-------------|
| prompt              | 0.9         |
| prompt + Bonjour    | 0.9         |
| prompt + Bonjour le | 0.9         |

- **TD (Temporal Difference) residual:**

$$\delta_t = r_{t+1} + \gamma V(s_{t+1}) - V(s_t).$$

- **GAE (Generalized Advantage Estimation):**

$$A_t^{GAE} = \delta_t + \gamma\lambda\delta_{t+1} + (\gamma\lambda)^2\delta_{t+2} + \dots$$

- **TD (Temporal Difference) residual:**

$$\delta_t = r_{t+1} + \gamma V(s_{t+1}) - V(s_t).$$

- **GAE (Generalized Advantage Estimation):**

$$A_t^{GAE} = \delta_t + \gamma\lambda\delta_{t+1} + (\gamma\lambda)^2\delta_{t+2} + \dots$$

$$A_t = \delta_t + \gamma\lambda A_{t+1}$$

## Monte Carlo target

$$L_{\text{value}}(\theta) = \frac{1}{N} \sum_t (V_{\theta}(s_t) - G_t)^2$$

## TD (Temporal Difference) target

$$L_{\text{value}}(\theta) = \frac{1}{N} \sum_t (V_{\theta}(s_t) - [r_{t+1} + \gamma V_{\text{old}}(s_{t+1})])^2$$

## GAE (Generalized Advantage Estimation) target

$$L_{\text{value}}(\theta) = \frac{1}{N} \sum_t (V_{\theta}(s_t) - [A_t^{GAE} + V_{\text{old}}(s_t)])^2$$

# Reward model vs. Value model

## 1. The reward model learns a function of full sequences

Reward model:

$$R(x, y)$$

Input:

(prompt, full completion)

Example:

input

reward

(Hello, "Bonjour le monde")

0.9

(Hello, "Salut le chien")

0.1

It **never** sees prefixes.

# Reward model vs. Value model (cont'd)

## 2. The value model sees prefixes

Value model input:

$$s_t = (x, a_1, \dots, a_{t-1})$$

Examples during training:

| prefix              | label $G_t$ |
|---------------------|-------------|
| prompt              | 0.9         |
| prompt + Bonjour    | 0.9         |
| prompt + Bonjour le | 0.9         |

But across many trajectories the same prefix can lead to **different final rewards**.

Example dataset:

# Reward model vs. Value model (cont'd)

| prefix  | completion                         | reward |
|---------|------------------------------------|--------|
| Bonjour | Bonjour le monde                   | 0.9    |
| Bonjour | Bonjour le chien mange une voiture | 0.1    |

So the value model learns

$$V(\text{Bonjour}) \approx 0.5$$

which is the **expected reward of continuations under the policy**.

### 3. The reward model cannot compute this expectation

The reward model only knows

$$R(x, y)$$

for a **specific finished sequence**.

It does not model

$$\mathbb{E}_{\pi}[R(x, y) \mid s_t]$$

which is what the value function must approximate.

---

### 4. The value function depends on the policy

This expectation depends on the **current policy**.

As the policy improves:

prefix → better completions

so

expected reward increases

The value model must be **updated continuously**.

The reward model stays fixed.

## 5. Another practical reason

Reward models are usually:

- trained with preference ranking loss
- not calibrated to produce consistent numeric expectations

Value models are trained with **regression to returns**, which matches the RL objective.

---

### Intuition

Think of it like writing an essay.

Reward model:

"How good is this finished essay?"

Value model:

"Given what is written so far, how good do I expect the final essay to be?"

These are different predictions.

---

### Key takeaway

Even though the **same final reward supervises the value model**, the value function learns the **expected reward of partial prefixes under the current policy**, which the reward model cannot represent.

# PPO (stable updates + token-level credit)

## A. PPO idea #1: do not move the policy too far (the ratio + clipping)

Define the ratio:

$$\rho_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\text{old}}(a_t \mid s_t)} = \exp(\log \pi_\theta - \log \pi_{\text{old}}).$$

PPO optimizes a clipped objective that prevents  $\rho_t$  from leaving  $[1 - \epsilon, 1 + \epsilon]$  in a way that would create overly large updates.

- $\pi_{\text{old}}$  is the policy that generated the batch.
- You can do multiple gradient steps on the same batch because importance ratios correct for drift.
- Clipping creates a trust region that stabilizes training.

## B. PPO idea #2: a learned value function gives token-level baselines

Value model predicts:

$$V(s_t) \approx \mathbb{E}[G_t \mid s_t].$$

Then advantage can be computed with TD or GAE (you can pick one to focus on):

- **TD (Temporal Difference) residual:**

$$\delta_t = r_{t+1} + \gamma V(s_{t+1}) - V(s_t).$$

- **GAE (Generalized Advantage Estimation):**

$$A_t^{GAE} = \delta_t + \gamma\lambda\delta_{t+1} + (\gamma\lambda)^2\delta_{t+2} + \dots$$

- In single-final-reward RLHF, the final reward enters at the end, and the value function propagates it backward through advantages.
- PPO has two losses: **policy loss** (clipped) and **value loss** (fit  $V$ ).

# PPO with Temporal-Difference (TD) error

## Final advantage

$$A_t = r_{t+1} + \gamma V(s_{t+1}) - V(s_t)$$

This differs across tokens.

---

## Meaning

Was this specific action better or worse than expected at this timestep?

- Positive  $\rightarrow$  reinforce this token
- Negative  $\rightarrow$  suppress this token

# PPO with Generalized Advantage Estimation (GAE)

## Final advantage

$$A_t = \delta_t + \gamma\lambda\delta_{t+1} + (\gamma\lambda)^2\delta_{t+2} + \dots$$

where

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t).$$

---

## Meaning

A smoothed measure of how much better this action was, accounting for both immediate and future prediction errors.

- $\lambda$  controls how far the reward propagates backward.
- Provides smoother and lower-variance credit assignment.

# Core PPO policy objective (clipped)

Define the ratio:

$$\rho_t(\theta) = \frac{\pi_{\theta}(a_t \mid s_t)}{\pi_{\text{old}}(a_t \mid s_t)}.$$

The PPO clipped objective is:

$$L^{\text{policy}}(\theta) = \mathbb{E} \left[ \min \left( \rho_t(\theta) A_t, \text{clip}(\rho_t(\theta), 1 - \epsilon, 1 + \epsilon) A_t \right) \right]$$

This is what we maximize.

**Thank you!**